CSE 237C Project 1

Logan Shehane

https://canvas.ucsd.edu/courses/77840/external_tools/6633
Project: FIR Filter Design — pp4fpgas 0.0.1 documentation

Question 1

a,b)

Design	Latency	II	BRAM	DSP	LUT	FF	MHz (Throughput)
FIR 11	19	1	0	2	383	733	1.064

Question 2

a)

Only coef_t and acc_t change. The sample interface and history remain 32-bit signed integers. All three designs pass the 600-sample golden test and the independent impulse test.

Design	Latency	II	BRAM	DSP	LUT	FF	MHz
16,32	134	135	1	1	254	255	1.104
5,16	134	135	1	1	239	229	1.104
6,16	134	135	1	1	239	229	1.104
8,16	134	135	1	1	239	229	1.104

b)

The minimum widths that match the supplied golden output are coef_t = ap_int<5> and acc_t = ap_int<16>. Coefficients range from -11 to 11, four signed bits represent only -8 to 7. The golden outputs range from -22,795 to 19,101, exceeding a signed 15-bit range. Actual one-bit-smaller tests fail: coef 4 gives -600 instead of 1,000 at sample 0; acc 15 gives 15,932 instead of -16,836 at sample 173. DSP usage remains the same because

Question 3

a-c)

The baseline FIR uses automatic loop pipelining.

Design	Latency	II	BRAM	DSP	LUT	FF	MHz
baseline	134	135	1	1	239	229	1.104
off	513	514	1	1	198	112	0.299
pipe1	134	135	1	1	239	229	1.104
pipe2	261	262	1	1	237	148	0.569
pipe3	388	389	1	1	254	148	0.383
pipe4	515	516	1	1	228	137	0.289
pipe16	515	516	1	1	228	137	0.289

d)

I set the pipeline value to 256. This value is way too high, and the resulting values were as follows:

pipe256	515	516	1	1	228	137	0.289

Using any pipeline number above 4 will have no impact on performance as the loop latency is 4.

e)

It appears that the baseline (automatic) numbers are the same as Pipeline 1. This is the most efficient latency and II value. The default is to pipeline once.

Question 4

a,b)

Moving the zeroth tap conditional out of the loop lowers the latency and II by 1 and reduces the BRAM by 1, while raising the LUT, FF, and clock speed.

Design	Latency	II	BRAM	DSP	LUT	FF	MHz
Conditional / auto	134	135	1	1	239	229	1.104

Design	Latency	II	BRAM	DSP	LUT	FF	MHz
Conditional / no pipeline	513	514	1	1	198	112	0.299
Hoisted / auto	134	135	1	1	214	130	1.104
Hoisted / no pipeline	636	637	1	1	159	57	0.241

Hoisting the conditional out of the loop reduces the control logic and reduces the resources, while also improving performance. Auto pipelining gives identical latency regardless of hoisting because the scheduler puts the mux into the pipeline stage, but hoisting saves on FFs and LUTs.

Question 5

a)

The loop can be partitioned to separate the dot product (accumulation) and the history (shift register) so that they can be done (relatively) in parallel.The dot product reads the updated history, so the loops have a real dependency and splitting alone does not make them concurrent.

b)

When splitting loops without unrolling or pipelining the two sequential loops will run back to back, relatively doubling latency.

Design	Latency	II	BRAM	DSP	LUT	FF	MHz
Baseline	134	135	1	1	214	130	1.104
Split Loops	267	268	1	1	273	128	0.556

c)

Split / Unroll 1	267	268	1	1	273	128	0.556
Split / Unroll 4	190	191	0	52	9684	11504	0.750
Split / Unroll 16	120	121	0	160	21517	17893	1.172

d)

Loop unrolling helps reduce latency at the cost of resources. When you unroll a loop, it creates more paths on the hardware for the operations to happen concurrently. This can be beneficial in

reducing the time, however it costs space on the board. When you unroll, there are then more operations that can occur at the same time, hence the pipelining is used. Loop unrolling provides spatial concurrency (the operations aren't happening on the same adders or multipliers. Loop pipelining provides the temporal concurrency that allows those operations (across different loop iterations) to overlap. Unrolling and looping can be used in tandem to gain speed while using less resources.

Question 6

a)

When doing memory partitioning, the arrays c and shift_reg get partitioned. In the original design the arrays are mapped to BRAM (block ram), which has limited accesses in a clock cycle. Partitioning divides the arrays between multiple memory banks (block or cyclic) or moves them into FF (flip flop registers) with the complete mode. Having the data in FFs allows for more read-writes per cycle, but uses more limited resources (FFs, LUTs, and multiplexers) instead of BRAM which although plentiful is slow.

Design	**Latency**	**II**	**BRAM**	**DSP**	**LUT**	**FF**	**MHz**
Baseline	134	135	1	1	214	130	1.104
Unroll 4	190	191	0	52	9684	11504	0.750
Unroll 4 Block 4	209	210	0	4	1803	975	0.676
Unroll 4 'c'	190	191	0	52	9686	11504	0.750
Unroll 4 complete	76	77	0	4	11675	2427	1.854
Unroll 4 Cyclic 4	77	78	0	4	564	394	1.854
Unroll 4 'shift reg'	77	78	0	4	562	394	1.854

b)

Disabling loop unrolling drastically increases the latency and II, while there is less difference in the

Cyclic 4	267	268	0	1	505	273	0.577
Unroll 4 Cyclic 4	77	78	0	4	564	394	1.854

Question 7

When optimizing a board there are two main things a designer can optimize for: output time and resources. The two outputs are inversely correlated, a board optimized for good output time will utilize more resources, whereas a board that is optimized for less resources will run far slower.
To optimize for resources, the baseline code is all that is needed. I specified a pipeline of 1 however this should be a default.
To optimize for time, I did complete memory partitioning and a pipeline of 1, with the loop unrolled
I've designed two boards to compare:

Design	**Latency**	**II**	**BRAM**	**DSP**	**LUT**	**FF**	**MHz**
Resource Efficient	134	135	1	1	214	130	1.104
Time Efficient	1	1	0	2	3036	2082	167.224

b)

The resource usage is much higher than the baseline due to the placementment of all the data in FF and LUT as opposed to BRAM, however this lets us complete all the operations in one cycle. Array partitioning alone (without unrolling the loops) does not speed anything up because the loop logic still occurs every cycle. Unrolling alone is bottlenecked by BRAM because you can only perform 1-2 reads or writes per cycle from the BRAM.

Question 8

https://github.com/lshehaneucsd/CSE237C_Proj1

Contributors

lshehaneucsd	Logan Shehane

References

I was originally trying to partner with Diwakar Basak, however we did our work separately. We did compare notes.
I chatted with Harini Sabapathy and Abhishek Asi, as I intend to do project 2 with them. Since we had already completed Project 1 separately, we are doing separate submissions.

I used AI to help research and understand the questions. The code for the assignment was written by hand, though I wrote a tool to automate running the results with some assistance from AI
Google. (2026). *Gemini* (Gemini3.8-Flash) [Large language model]. https://gemini.google.com/